CMAN :=

Task 1 := The dominant kernel is the 3x3 CONV2D dot product the inner loop of an im2col style convolution. It runs inside conv2D_im2col and accounts for 80.6% of total software runtime. The hardware design replaces exactly this kernel with 4 parallel MAC processing elements.

Kernel dimensions :=

Parameter	Value.
Kernel spatial size	3x3 = 9 taps
I/p channels	1
O/p channels/PEs	4 (NUM_PE = 4)
I/p feature Map	32x32 = 1024 Pixels
O/p feature Map	(32-3+1) x (32-3+1) = 30x30 = 900 o/p patches

Data types :=

Data	TYPE	size
Input pixels	INT8 signed	1 byte each
Weights	INT8 signed	1 byte each
Accumulator	INT32 signed	4 bytes each

The INT32 accumulator is needed because the maximum partial sum is $9 \times 127 \times 127 = 145,161$, which overflows INT8 and INT16.

Task 2 := Each MAC operation = 2 FLOPS

Each PE does 9 MACs → 18 FLOPS

4 PEs running in parallel → 4 x 18 = 72 FLOPs per patch

Across all 900 output patches for one full 32x32.

Total FLOPs per invocation = 641800 FLOPs.

Task 3 :=

Lower Bound (No data reuse) :=

Every time needs a weight or a pixel, it goes all the way to off chip memory and fetches it even if it just used that same value one patch ago.

for each of the 900 output patches.

pixel for this patch loaded is 9 pixels x 1 byte x 900 patches

weights for all 4 PEs is 4 x 9 weights x 1 byte x 900 patches

Bytes_lower = (9+36) x 900 = 45 x 900 = 40,500 bytes

Lower bound = 40,500 bytes, this gives the lowest possible AI.

Upper Bound (perefect on-chip data reuse for weights)

Bytes_upper = 36 + 8100 = 8,136 bytes

Upper Bound = 8,136 bytes, this gives the highest possible AI.

Task 4 :=

Arithmetic Intensity for both Bounds.

AI = Total FLOPs ÷ Total Bytes.

From Task 2, FLOPs = 64,800

Bound

Low (No reuse) ← 64 800 FLOPs, 40,500 Bytes

AI = 1.60 FLOP/byte.

Upper (weight reuse) ← 64,800 FLOPS, 8136 Bytes

AI = 7.96 FLOP/byte.

Sky130A Platform at 100MHz.

Peak compute = 4 PEs x 1 Mac/cycle X 100 MHz X 2 FLOPS/MAC
= 800 MOPS

Peak BW = 330 MB/s (AXI4-Lite 32-bit bus)

Ridge point = 800 ÷ 330 = 2.42 FLOP/byte

So, AI = 1.60 is left of the ridge (2.42), it is a Memory bound with no reuse.

AI = 7.96 is right of the ridge (2.42), it is a compute bound with weight reuse.

At AI = 1.60, attainable performance = 330 X 1.60
= 528 MOPS

At AI = 7.96, attainable performance = Min (330X 7.96, 800)
= 800 MOPS (compute ceiling)

TASK 5 :=

In my Design, limited by hardware interface bandwidth, on-chip memory Bandwidth.

Hardware interface bandwidth (AXI4-Lite):

One patch takes 120 clock cycles. Only 9 of those cycles are actual MAC computation that is 7.5% utilisation. The remaining 111 cycles are AXI4 Lite bus overhead while the MAC units sit idle waiting for data. The design is not short on compute or chip memory it is starved by the interface.

What is the single highest leverage change?

Replace the per patch sequential AXI writes with a deep streaming pixel FIFO. This collapses pixel delivery from ~111 bus overhead cycles down to ~9 FIFO cycles per patch cutting total patch latency from 120 cycles to ~30 cycles and raising throughput from 60MOPS to ~240MOPS, so 4X improvement with no changes to the MAC array.